

Relatório técnico do cbuid

Trabalho Prático 1 - MAC0216 - Equipe 22

Integrantes:

Thiago Assumpção Baisch

Gustavo Yukio Yamani

Matheus Guimarães Olegario

Matheus Moreira Cabral

Repositório do projeto: <https://github.com/omatheusolegario/mac0216-cbuild-equipe22>

1 Introdução

O cbuid é uma ferramenta de linha de comando para construir e executar projetos escritos em C. Desenvolvemos a solução em Bash, com comandos para compilar, executar, limpar os arquivos gerados, reconstruir o projeto e consultar suas informações. A ferramenta também registra as operações e oferece modos de saída mais detalhados para acompanhar o que está acontecendo.

A ideia foi reunir essas tarefas em uma interface simples. O usuário informa o comando e a pasta do projeto; o cbuid encontra as fontes, prepara os diretórios e chama as ferramentas necessárias. O GCC realiza a compilação e a ligação dos objetos. O GNU Make ajuda a verificar se as dependências mudaram, permitindo reaproveitar arquivos objeto que continuam atualizados.

O projeto foi dividido em quatro frentes. Antes da implementação, combinamos os nomes das funções, as variáveis compartilhadas e os códigos de retorno. Isso permitiu desenvolver as partes separadamente e testar funções mesmo quando os outros módulos ainda não estavam prontos.

Integrantes e responsabilidades

Integrante	Participação principal
Gustavo Yukio Yamani	Interface de comandos, configuração, descoberta dos arquivos e organização do fluxo principal, em <code>cbuid</code> e <code>lib/project.sh</code> .
Matheus Guimarães Olegario	Compilação incremental e ligação, em <code>lib/build.sh</code> ; integração das branches e ajustes na captura de erros.

Integrante	Participação principal
Thiago Assumpção Baisch	Execução, limpeza e reconstrução, em <code>lib/actions.sh</code> , com verificações de caminhos e propagação de falhas.
Matheus Moreira Cabral	Logs, informações do projeto, relatório de operações e mensagens de verbose e debug, em <code>lib/observability.sh</code> , video da apresentação.

2 Arquitetura da solução

Organizamos o cbuild em um script de entrada e quatro módulos. O script principal interpreta a chamada, prepara o projeto e encaminha o comando. Cada módulo concentra um grupo de funções, evitando que a lógica de compilação, execução e registro fique toda misturada.

Componente	Responsabilidade
cbuild	Interpretar argumentos, carregar os módulos, coordenar a operação e capturar stderr.
lib/project.sh	Ler a configuração, resolver caminhos e descobrir fontes e cabeçalhos.
lib/build.sh	Decidir quais fontes recompilar, gerar objetos e ligar o executável.
lib/actions.sh	Executar o programa, limpar a compilação e reconstruir o projeto.
lib/observability.sh	Registrar operações, mostrar informações e produzir o relatório textual.

Como os módulos se conectam

Os módulos são carregados com `source`, no mesmo processo Bash. Suas funções passam a compartilhar o contexto do projeto, sem precisar reinterpretar os argumentos ou ler novamente a configuração.

Variável	Significado
PROJECT_DIR	Caminho absoluto da raiz do projeto.
BUILD_DIR e LOG_DIR	Pastas <code>build/</code> e <code>logs/</code> , dentro da raiz.

Variável	Significado
EXECUTABLE	Caminho completo do executável em <code>build/</code> .
SOURCE_FILES e HEADER_FILES	Arrays com os caminhos dos arquivos <code>.c</code> e <code>.h</code> .
OPT_LEVEL	Nível de otimização solicitado, de 0 a 3.
VERBOSE, DEBUG e REPORT	Opções de mensagens adicionais e geração de relatório.

Fluxo de uma chamada

1. Criar o temporário para os erros e configurar os tratamentos de saída.
2. Interpretar e validar os argumentos com `parse_args`.
3. Carregar os módulos e preparar o projeto com `project_init`.
4. Iniciar o registro com `log_begin`.
5. Executar a função escolhida por `dispatch_command` e guardar seu status.
6. Finalizar o registro com `log_end`.
7. Gerar `logs/report.txt`, se `--report` estiver ativo, e devolver o resultado.

Os artefatos de compilação ficam em `build/`, enquanto o histórico permanece em `logs/`. Essa separação é importante para que `clean` não apague os registros. O projeto de exemplo está em `tests/projeto_teste01`, com três fontes e dois cabeçalhos.

3 Módulos implementados

3.1 Entrada e controle do programa

O arquivo `cbuild` começa definindo os valores padrão: diretório corrente como raiz do projeto, nível de otimização 0 e opções adicionais desativadas. A função `usage` mostra a sintaxe e os comandos disponíveis.

`parse_args` exige que o primeiro argumento seja `build`, `run`, `clean`, `rebuild` ou `info`. Depois processa as opções. Comandos desconhecidos, argumentos extras, valores ausentes e níveis de otimização inválidos interrompem a chamada com código 2. Quando uma opção de valor é repetida, o último valor aceito pelo parser prevalece; a existência do diretório é verificada depois, na preparação.

`load_modules` usa `BASH_SOURCE` para localizar a própria ferramenta e carregar os arquivos de `lib/`. Assim, a localização dos módulos não depende da pasta de onde o usuário chama o programa. `dispatch_command` faz o encaminhamento para a função correspondente ao comando.

`main` organiza as etapas e guarda o código de retorno antes de finalizar o log. Se a operação falhar, esse resultado é devolvido após a finalização normal. Uma falha no log ou na geração do relatório faz a ferramenta retornar 1 imediatamente, podendo substituir o código original. Esse é um comportamento da versão atual que pode ser refinado.

3.2 Configuração e preparação do projeto

O módulo `project.sh` contém `read_executable_name` e `project_init`. A primeira lê `.cbuild.conf` como texto, sem executar seu conteúdo. O formato aceito é:

```
EXECUTABLE=programa
```

O valor deve aparecer sem aspas e sem espaços ao redor de `=`. O primeiro caractere precisa ser uma letra ou número; os seguintes também podem ser ponto, hífen ou sublinhado. Definições duplicadas, nomes inválidos e linhas desconhecidas provocam erro. Linhas vazias e comentários iniciados por `#` na primeira coluna são ignorados. Sem arquivo ou sem a chave, o nome padrão é `programa`.

`project_init` converte a raiz em caminho absoluto com `cd` e `pwd -P`, define o contexto e cria `build/` e `logs/`. Ela rejeita a raiz `/`, diretórios inexistentes e links simbólicos nas posições de

`build/` e `logs/` .

A descoberta usa `find` , excluindo as duas pastas de saída com `-prune` . A opção `-print0` e a leitura com `read -r -d ''` preservam cada caminho como um item. A listagem passa por um temporário criado com `mktemp` , permitindo verificar se a busca falhou antes de ler seu resultado. Depois, os caminhos são separados nos arrays de fontes e cabeçalhos.

A busca é recursiva em toda a raiz, não apenas em `src/` . Por isso, a pasta selecionada deve representar um projeto que gera um único executável. Várias fontes com funções `main` diferentes podem causar erro na ligação.

3.3 Compilação incremental

O módulo `build.sh` implementa `build_project`. A função verifica se GCC e Make estão disponíveis e se existe pelo menos uma fonte. Essas exigências ficam na construção, para que comandos como `info` e `clean` não dependam da presença do compilador.

Para cada fonte, são definidos os caminhos do objeto e do arquivo de dependências. A estrutura relativa do projeto é preservada: `src/soma.c` gera `build/src/soma.o` e `build/src/soma.d`. Essa escolha evita que fontes de mesmo nome em subdiretórios diferentes disputem o mesmo arquivo objeto.

Decisão de recompilar

O arquivo `build/OPT_LEVEL.txt` guarda o nível da última construção concluída. Se ele não existir ou o nível solicitado mudar, todos os objetos são recompilados. O marcador antigo é removido antes do processamento e só é gravado novamente após o sucesso. Assim, uma falha no meio da mudança de otimização não deixa a tentativa incompleta validada pelo marcador.

A ausência do `.o` ou do `.d` de uma fonte também força sua compilação. Quando os dois existem e o nível não mudou, a ferramenta consulta o Make:

```
make -Rr -f - -q "$caminho_obj"
```

Uma regra mínima é fornecida por entrada padrão, junto com a inclusão do arquivo `.d`. O Make apenas responde se o objeto precisa ser atualizado: retorno 0 significa atualizado; 1 significa que precisa recompilar; outros valores são tratados como falha na consulta.

Compilação e dependências

```
gcc -c -I "$PROJECT_DIR/include" -MMD -MP \  
"$arquivo_fonte" -O"$OPT_LEVEL" -o "$caminho_obj"
```

`-c` gera o objeto sem fazer a ligação. `-I` acrescenta `include/` à busca de cabeçalhos. `-MMD` gera as dependências não classificadas como cabeçalhos de sistema, incluindo inclusões indiretas reconhecidas pelo compilador. `-MP` acrescenta alvos auxiliares para cabeçalhos, evitando determinados erros do Make quando uma dependência antiga deixa de existir. Se um cabeçalho ainda necessário estiver ausente, o GCC continua indicando erro.

Ligação do executável

Os objetos correspondentes às fontes atuais são guardados em um array e passados ao GCC. Isso evita incluir automaticamente objetos antigos de fontes removidas ou renomeadas. Esses arquivos podem permanecer em `build/` até a limpeza, mas deixam de participar da ligação.

A ligação é executada em todo `build` bem-sucedido, mesmo quando os objetos não mudaram. Portanto, a economia incremental está na etapa de compilação. Ao terminar, o programa salva o nível de otimização e informa o caminho do executável. Falhas de compilação ou ligação interrompem a operação.

3.4 Execução limpeza e reconstrução

O módulo `actions.sh` reúne as três funções implementadas por Thiago. `run_project` verifica se o caminho do executável foi definido, se existe, se é um arquivo regular e se tem permissão de execução. Depois chama `"$EXECUTABLE"`, preservando a entrada padrão e a saída normal. Se o programa terminar com erro, a função mostra uma mensagem e devolve seu código.

`run` não faz uma compilação automática nem verifica se as fontes foram alteradas. Também não muda o diretório corrente para a raiz do projeto. Um programa que utiliza caminhos relativos continua dependendo da pasta de onde a ferramenta foi chamada. As verificações de arquivo e permissão não substituem uma tentativa de execução: o arquivo ainda pode falhar por outros motivos.

`clean_project` confere os caminhos antes de executar `rm -rf -- "$BUILD_DIR"`. Ela recusa a raiz do sistema, o próprio diretório do projeto e caminhos fora dele. Também impede a remoção quando os logs estão dentro do diretório de compilação. Se a pasta não existir, a função retorna sucesso; se o caminho existir sem ser um diretório, retorna erro. Remover a pasta inteira evita deixar arquivos ocultos para trás.

`rebuild_project` chama `clean_project` e depois `build_project`. Se a limpeza falhar, a construção não começa. O código de retorno de cada falha é preservado imediatamente. Não há uma segunda implementação da compilação dentro de `rebuild`.

3.5 Logs informações e relatório

O módulo `observability.sh` fornece `log_begin`, `log_end`, `show_info`, `generate_report`, `verbose_msg` e `debug_msg`. O início e o fim do log envolvem a operação escolhida; um `rebuild` aparece como um único registro.

A captura de `stderr` fica no script principal. Um descritor auxiliar preserva o destino original, e `tee` envia as mensagens ao terminal e a um temporário exclusivo. Antes de ler esse arquivo, `fecha_leitura_erros` restaura `stderr` e espera `tee` terminar. O tratamento de saída com `trap` remove o temporário ao encerrar.

Cada linha de `logs/operations.log` contém data, horário, comando, status, duração, código de retorno e mensagens capturadas. A duração usa `date +%s%N` e é apresentada em segundos com três casas decimais. Um aviso do GCC pode aparecer no campo de erros mesmo quando a operação termina com sucesso: o status depende do código de retorno.

`show_info` conta fontes e cabeçalhos, soma suas quebras de linha com `wc -l`, mede o binário com `wc -c` e procura no histórico as últimas construções e execuções bem-sucedidas. Quando um dado não existe, mostra que está indisponível. `generate_report` reescreve `logs/report.txt` com totais de operações, sucessos, falhas, compilações bem-sucedidas e execuções bem-sucedidas. Esse arquivo é diferente do presente relatório técnico.

4 Decisões de projeto

Mudanças em relação ao planejamento

Proposta inicial nos documentos de divisão	Solução integrada
Opção <code>--0</code> , com níveis 0, 1 e 2.	A interface aceita <code>--0</code> , com níveis 0, 1, 2 e 3.
<code>record_event</code> chamado por <code>build</code> e <code>run</code> .	A função foi retirada; as datas são obtidas de <code>operations.log</code> .
Integração final atribuída a Gustavo.	Gustavo implementou o fluxo principal; Olegario relata ter integrado as branches e corrigido pendências.
Registro de <code>stderr</code> concentrado na frente de logs.	O script principal captura as mensagens; <code>log_end</code> as incorpora ao histórico.
Relatório textual previsto com estatísticas do projeto.	<code>report.txt</code> resume operações; as informações dos arquivos e do binário ficam em <code>info</code> .

5 Exemplos de uso

Preparação e primeira execução

A ferramenta foi desenvolvida para Linux com Bash, GCC e GNU Make. Ela também usa utilitários como `find` , `mktemp` , `tee` , `date` , `sed` , `awk` , `grep` e `wc` . A medição em nanossegundos depende de suporte a `date +%s%N` , disponível no GNU `date`.

Os exemplos abaixo partem da pasta da ferramenta. A chamada por `bash` dispensa alterar a permissão do script. Para usar `./cbuild` , primeiro execute `chmod +x cbuild` .

```
bash cbuild build --dir tests/projeto_teste01 --verbose
bash cbuild run --dir tests/projeto_teste01
```

No exemplo, informar `8 3` produz soma 11 e subtração 5. Repetir o `build` sem alterações reaproveita os três objetos, embora a ligação seja feita novamente.

Otimização e mensagens adicionais

```
bash cbuild build --dir tests/projeto_teste01 --0 2
bash cbuild build --dir tests/projeto_teste01 --verbose --debug
```

`--verbose` mostra as etapas, enquanto `--debug` mostra decisões e valores internos. As opções são independentes. O nível padrão é 0 em cada chamada: depois de construir com `--0 2`, omitir a opção em outro `build` solicita nível 0 e provoca nova compilação dos objetos.

Informações histórico e limpeza

```
bash cbuild info --dir tests/projeto_teste01 --report
cat tests/projeto_teste01/logs/report.txt
cat tests/projeto_teste01/logs/operations.log
bash cbuild clean --dir tests/projeto_teste01
bash cbuild rebuild --dir tests/projeto_teste01 --0 2
```

`--report` pode acompanhar qualquer comando válido. O relatório é gerado depois do registro, incluindo a operação que o solicitou. `clean` remove `build/` e preserva `logs/`; `rebuild` limpa e compila novamente.

Nome personalizado e falhas esperadas

Para gerar `build/calculadora`, crie `.cbuild.conf` na raiz do projeto com `EXECUTABLE=calculadora`, sem aspas. Chamar `run` antes de uma construção válida retorna erro de executável ausente. Informar `--0 4` ou uma opção desconhecida é rejeitado. A ferramenta ainda não oferece argumentos extras para o programa executado nem opções para bibliotecas adicionais.

6 Relatos individuais

6.1 Gustavo Yukio Yamani

Parte: integração, preparação do projeto e gerenciamento do fluxo do cbuild

Interface e interpretação dos argumentos

Criei `usage` para mostrar a mensagem de uso do programa. Em seguida, implementei `parse_args`, responsável por identificar o comando solicitado e processar as opções. As informações recebidas ficam em variáveis utilizadas pelo restante do programa, como `COMMAND`, `VERBOSE` e `DEBUG`.

A validação impede a continuação quando os argumentos são inválidos, como comando inexistente, ausência de comando ou utilização de mais de um comando. Quando há mais de um `--dir`, o último valor aceito pelo parser prevalece. A existência desse diretório é conferida na preparação do projeto.

Adicionei um trecho temporário em `cbuild` para exibir os valores obtidos por `parse_args`. Isso também foi útil para verificar a formatação de `usage`. Testei chamadas com `build`, `run --verbose` e `info --debug --report`, além de diretório com espaços. Também testei ausência de comando, comando errado, dois comandos, `--dir` sem valor e opção desconhecida.

Configuração e descoberta dos arquivos

Criei `lib/project.sh` e implementei `read_executable_name` para obter o nome a partir de `.cbuild.conf`. Sem arquivo ou sem uma definição de `EXECUTABLE`, é utilizado o nome padrão `programa`. Se houver um valor inválido ou mais de uma definição, a função retorna erro. A leitura usa `IFS= read -r`, preservando espaços e barras invertidas.

Depois implementei `project_init`. Antes de tudo, o diretório informado é convertido para um caminho absoluto. Defini `PROJECT_DIR`, `BUILD_DIR`, `LOG_DIR`, `EXECUTABLE`, `SOURCE_FILES` e `HEADER_FILES`, que ficam disponíveis para os outros módulos.

A implementação do `find` foi a parte mais problemática e complexa de `project_init`, tanto pelo funcionamento quanto pela sintaxe. Usei um arquivo temporário criado com `mktemp` para verificar se a busca foi executada corretamente. Assim, uma falha não é confundida com uma busca que não encontrou arquivos. Os caminhos são separados com `-print0` e armazenados em arrays.

Para testar essa parte, fiz um teste com arquivos C vazios, sem envolver compilação. Ele falhou duas vezes por causa do `find`, que tive que consertar.

Organização do fluxo principal

Implementei `load_modules` com `source`, para disponibilizar as funções no mesmo processo, e `dispatch_command`, para chamar a função correta. A `main` organiza a interpretação, a preparação, o início do log, o comando e a finalização.

O código de retorno é guardado antes das etapas de finalização para não se perder com a execução de outro comando. O relatório é gerado depois do log, para ter acesso ao registro completo da operação. Na versão integrada, esse fluxo também inclui a captura de erros feita com `tee`.

6.2 Matheus Guimarães Olegario

Parte: compilação, ligação, dependências e integração das branches

Fiquei responsável pela implementação do comando `build` do nosso `cbuild`, tendo que lidar com compilação, ligação de objetos e compilação incremental. A função recebe os caminhos, as fontes e o nível de otimização preparados pela parte do Gustavo.

Primeira abordagem e dificuldades

A grande dificuldade foi achar um jeito de fazer a compilação incremental funcionar. A primeira ideia foi comparar as datas dos arquivos `.c` e `.o` e procurar dependências com `grep` dentro das fontes. Essa abordagem logo falhava para dependências de dependências. Nomes com espaços e outros caracteres também não eram tratados corretamente nas minhas tentativas com `grep`, `sed` e `awk`, porque apareciam muitos casos diferentes para resolver.

Comecei a andar no caminho certo ao delegar essas tarefas para as ferramentas que já sabem fazê-las. Usei `-MMD -MP` no GCC para obter as dependências de cada `.c`. Depois, usei o GNU Make no modo de consulta para saber se alguma delas tinha sido modificada desde a última compilação do objeto. Isso resolveu a parte das dependências encadeadas dentro dos casos suportados pela ferramenta.

Decisões na implementação

Preservei o caminho relativo das fontes dentro de `build/`. Se uma fonte está em `src/`, o objeto correspondente fica em `build/src/`. Assim, arquivos de mesmo nome em pastas diferentes não geram o mesmo objeto.

Também guardei em um array apenas os objetos correspondentes às fontes atuais. Na ligação, uso esse array, evitando que um objeto antigo de uma fonte removida entre no executável. Antes do laço principal, verifico se o nível de otimização mudou. O nível só é salvo em `OPT_LEVEL.txt` depois de uma construção bem-sucedida; se mudou, o marcador antigo é apagado antes da recompilação.

Integração e testes

Também fui responsável por juntar as partes das branches na `main`, corrigir erros e verificar o que estava pendente. O principal vilão foi capturar os erros em um temporário e, ao mesmo tempo, continuar mostrando tudo no terminal. Entender como usar `tee`, que executa em segundo plano nesse fluxo, e como evitar problemas de sincronismo foi desafiador.

A solução envolveu criar um novo canal, usar `mktemp` para separar os temporários das execuções e `trap` para removê-los no encerramento. Também foi necessário esperar o `tee` terminar antes de ler os erros.

Nos testes do desenvolvimento, verifiquei primeira compilação, repetição sem alterações, mudança de fonte e cabeçalho, remoção de `.d`, mudança de otimização, falha no meio da recompilação e recuperação, renomeação de fonte e caminhos com espaços. Os casos registrados no meu relato passaram.

Fazer a interligação das partes ajudou muito a entender a importância das aspas nas variáveis e do direcionamento de erros. Na parte de build, aprendi mais sobre Make e as opções do GCC. Uma melhoria seria tratar melhor casos como `$` e `#` nos caminhos, para ampliar os projetos que a ferramenta consegue aceitar.

6.3 Thiago Assumpção Baisch

Parte: execução, limpeza e reconstrução

Comecei fazendo o esqueleto de `run_project`, `clean_project` e `rebuild_project` no arquivo `actions.sh`.

Execução do programa

Criei `run_project` com as verificações de erro. Primeiro confiro se o caminho foi definido; depois, se existe, se corresponde a um arquivo e se tem permissão de execução. Prefiro deixar os `ifs` separados, para ficar claro o motivo de cada erro.

Depois fiz um `if` que executa `"$EXECUTABLE"`. Se funcionar, o status fica como 0. Se não funcionar, guardo o código e retorno esse erro. Tive um pouco de dificuldade com a sintaxe dos `ifs` e para entender como testar a execução dentro de um `if`, mas consegui.

Na primeira versão, também tinha uma chamada para `record_event`. Cheguei a separar a falha do programa da falha desse registro. Na versão final, essa função saiu: o registro ficou no fluxo principal, com `log_begin` e `log_end`. Para testar minha parte isoladamente, usei funções substitutas de mensagens e, naquela etapa, de `record_event`.

Limpeza e revisão dos caminhos

Pensei em `clean_project` como uma função que primeiro precisa ter certeza de que está apagando o lugar certo. Comecei verificando se `PROJECT_DIR` e `BUILD_DIR` estavam definidos. Depois coloquei verificações para evitar apagar a raiz `/`, o próprio diretório do projeto ou uma pasta fora dele.

Se a pasta não existir, a função retorna 0, porque o projeto já está limpo. Tive um pouco de dificuldade para entender como conferir se `BUILD_DIR` estava dentro de `PROJECT_DIR`. Também fiquei com receio de usar `rm -rf`, porque sabia que um caminho errado poderia apagar coisas importantes.

Eu tinha pensado em apagar somente `"$BUILD_DIR"/*`, mas isso poderia deixar arquivos ocultos. Então achei melhor apagar a própria pasta, que pode ser criada novamente. Também acrescentei uma verificação para o caso de o caminho apontar para um arquivo em vez de uma pasta. Se os logs estiverem dentro de `BUILD_DIR`, a limpeza é interrompida para não apagar o histórico.

A lógica de apagar era simples. A parte mais difícil foi pensar nos casos em que o caminho poderia estar errado. Acabou exigindo mais verificações do que eu tinha imaginado.

Reconstrução e testes com mocks

`rebuild_project` foi mais fácil, porque só precisa chamar `clean_project` e depois `build_project`. Se a limpeza falhar, guardo o status e retorno sem chamar a compilação. Não repeti a lógica de build dentro da função. Também precisei tomar cuidado com `status=$?`, guardando o valor antes de executar outro comando.

Na minha branch, `build_project` ainda não existia. Criei mocks de `clean` e `build`, que só retornavam os valores escolhidos, e uma variável `ordem` para conferir as chamadas. Testei sucesso das duas, falha de `clean` sem chamar `build` e falha de `build` depois de `clean` funcionar.

Fiquei confuso porque já tinha importado a `clean_project` original. Depois entendi que definir outra função com o mesmo nome substitui a anterior naquele processo de teste. Coloquei os mocks depois dos testes da função verdadeira. Essa parte foi mais fácil de programar; o mais chatinho foi entender os mocks e testar a ordem das funções.

6.4 Matheus Moreira Cabral

Parte: logs, estatísticas e relatórios

As funções da minha parte ficaram em `observability.sh`. Comecei fazendo `show_info`, chamada pelo comando `info` para mostrar as informações do projeto.

Informações e critério de contagem

A função exibe o número de arquivos, o número de linhas, o tamanho do executável e as datas da última compilação e execução. A contagem de arquivos considera as fontes `.c` e os cabeçalhos `.h` encontrados na preparação.

Fiz a contagem de linhas com `wc -l`. O critério é a presença de `\n`, então linhas vazias, comentários e diretivas de pré-processamento entram na contagem. Uma última linha sem `\n` não é contada. Quando uma informação não está disponível, a função informa isso e retorna normalmente.

Inicialmente o plano era fazer outra função, `record_event`, chamada depois de cada compilação e execução para guardar as datas. Decidi percorrer o log para obter essas informações. Assim, `record_event` não existe mais na versão final. Para as datas, são considerados os últimos registros bem-sucedidos de build ou rebuild e de run.

Início e finalização do log

`log_begin` é chamada antes do comando e recebe seu nome. Ela deixa esse valor salvo para `log_end` e usa `date` para guardar o horário inicial.

`log_end` é chamada depois e recebe o código de retorno do comando. Ela usa os dados salvos para calcular o tempo e acrescenta uma linha em `logs/operations.log`, com data, horário, comando, sucesso ou falha, duração, código e erros capturados.

Na integração final, a captura das mensagens ficou no script principal. Antes de ler o temporário, `log_end` chama a função que encerra essa captura e espera sua conclusão. A função também verifica falhas ao preparar o diretório, ler os erros e gravar o log.

Relatório de operações e mensagens

`generate_report` é chamada quando a opção `--report` está ativa. Ela prepara `operations.log`, criando o arquivo vazio se necessário, e usa seus registros para escrever `logs/report.txt`. Se não conseguir preparar ou processar os arquivos, retorna erro.

O relatório contém o total de operações, as operações bem-sucedidas e fracassadas, as compilações e as execuções. Na implementação final, os dois últimos números contam apenas as operações bem-sucedidas. Esse relatório resume o histórico; as estatísticas dos arquivos e do executável continuam no comando `info`.

As funções `verbose_msg` e `debug_msg` são as mais simples. Elas recebem uma mensagem e só a exibem quando o modo correspondente está ativo. Assim, os outros módulos podem chamar essas funções sem precisar repetir a verificação das opções.

A principal mudança de projeto registrada na minha parte foi usar o próprio log como fonte das datas, sem manter um segundo registro separado para build e run. Isso também mudou a forma como os outros módulos se conectavam à minha parte.

7 Resultados obtidos

A solução integrada reúne os cinco comandos previstos, configuração do executável, quatro níveis de otimização, mensagens adicionais, logs e relatório textual. A construção incremental reage a mudanças nas fontes e nas dependências efetivamente registradas pelo compilador. A limpeza mantém o histórico fora dos artefatos removidos.

Verificação da versão final

Além dos testes descritos nos relatos individuais, foi feita uma verificação complementar da versão do ZIP em Linux, usando uma cópia isolada de `tests/projeto_teste01` . A checagem `bash -n` passou no script principal e nos quatro módulos. Os resultados abaixo correspondem a essa verificação, não a uma declaração de cobertura de todos os casos do enunciado.

Cenário	Resultado observado
Primeira construção e execução	Sucesso. Com entrada 8 e 3, o programa produziu soma 11 e subtração 5.
Build repetido sem alterações	Os três objetos mantiveram suas datas. A ligação ocorreu novamente.
Alteração de <code>soma.c</code>	Apenas <code>soma.o</code> foi recompilado.
Alteração de <code>subtracao.h</code>	Apenas <code>main.o</code> mudou. A fonte <code>subtracao.c</code> não inclui esse cabeçalho.
Mudança para <code>--0 2</code>	Todos os objetos foram recompilados e a construção terminou com sucesso.
<code>info --report</code>	Mostrou as informações e concluiu a geração do relatório.
Limpeza	<code>build/</code> foi removido e <code>operations.log</code> permaneceu disponível.
Run depois de clean	Retornou 1, informando que o executável não foi encontrado.
Reconstrução	<code>rebuild</code> voltou a gerar o executável com sucesso.
Nome sem aspas na configuração	Gerou o executável <code>calculadora</code> .

Cenário	Resultado observado
Nome com aspas e opção desconhecida	Foram rejeitados, com códigos 1 e 2, respectivamente.

O teste do cabeçalho mostra que a decisão acompanha a dependência real, não apenas nomes parecidos. Mesmo implementando a função declarada em `subtracao.h`, `subtracao.c` não o inclui na versão entregue; apenas `main.c` aparece como dependente desse cabeçalho.

Alcance dos resultados

Os relatos registram testes com mocks, espaços nos caminhos, remoção de `.d` e recuperação de falhas. As pastas de testes individuais não estão no ZIP; nele existe apenas `tests/projeto_teste01`. Os testes relatados não constituem uma suíte automatizada disponível na entrega.

Na construção com otimização 2, o GCC emitiu um aviso sobre o retorno de `scanf` ser ignorado pelo exemplo. O build ainda terminou com código 0. Não foram medidos ganhos de tempo ou desempenho entre níveis de otimização; os resultados confirmam o comportamento funcional dos cenários verificados.

8 Considerações finais

O trabalho reuniu tarefas de shell, compilação e organização de código. A divisão por módulos permitiu desenvolver as partes separadamente, mas a integração mostrou a importância de combinar funções, caminhos e retornos desde o começo. As maiores dificuldades ficaram na descoberta de arquivos, nas dependências, na limpeza e na captura de erros.

A versão final executa o fluxo proposto nos cenários verificados. Ampliar os testes, atualizar a documentação e melhorar o tratamento das falhas são os próximos passos para tornar a ferramenta mais robusta.